

JJBike

1. Objetivo.

Construir um armazém de dados para que o administrador da empresa fictícia JJBike possa compreender seu negócio.

2. Modelagem.

Para criar o modelo da estrutura do banco de dados analítico (OLAP) com base no banco de dados transacional (OLTP), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

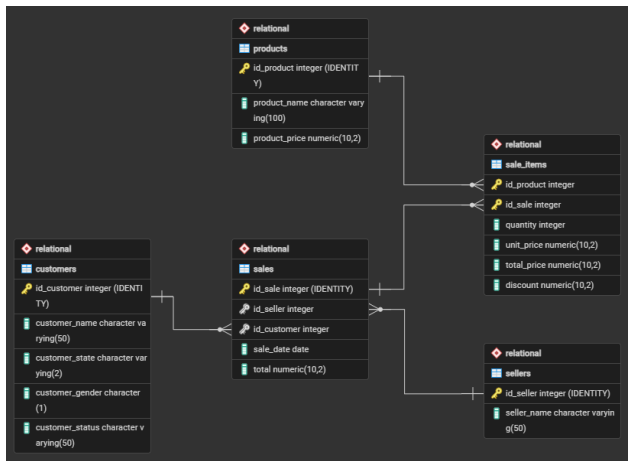
1. Como o objetivo é descrever o modelo de negócio, todas as tabelas do ambiente OLTP que possuem relações um para muitos (1:N) com a tabela *sales* (vendas), foram transformadas em dimensões:

- **Produto ('o quê?'):** *dim_product* (dimensão produto), que possui os atributos da tabela *products* (produtos) do ambiente OLTP (exceto *product_price* (preço do produto), pois isso fica na tabela fato), mais a adição de *validity_start_date* (data de início de validade) e *validity_end_date* (data de fim de validade) para versionamento;
- **Vendedor ('quem?'):** *dim_seller* (dimensão vendedor), que possui os atributos da tabela *sellers* (vendedores) do ambiente OLTP, mais a adição de *validity_start_date* e *validity_end_date* para versionamento;
- **Cliente ('quem?'):** *dim_customer* (dimensão cliente), que possui os atributos da tabela *customers* (clientes) do ambiente OLTP, mais a adição de *validity_start_date* e *validity_end_date* para versionamento.

2. Foi criada a dimensão tempo ('quando?'): *dim_time* (dimensão tempo), que possui o atributo data da tabela *sales* do ambiente transacional, mais os atributos derivados *time_day* (dia), *time_month* (mês), *time_year* (ano), *time_weekday* (dia da semana) e *time_quarter* (trimestre), criados a partir de *time_date* (data), para possibilitar hierarquia de dimensões nas análises;

3. No ambiente OLAP, a tabela fato *fact_sales* (fato vendas) consolidou a tabela de cabeçalho (*sales*) e a de detalhes (*sale_items* (itens de venda) (N:M entre vendas e produtos)), do ambiente OLTP, em um único nível de granularidade: o item de produto por venda. Possuindo sua Surrogate Key (SK) como Primary Key (PK), as SKs das dimensões como suas Foreign Keys (FK) e os atributos da tabela de ligação *sale_items* como métricas.

- A granularidade da tabela fato *fact_sales* foi definida ao nível de item de linha por venda, garantindo o maior nível de detalhamento possível para as análises do administrador da JJBike.



3. Ambiente Transacional (OLTP) - Microsoft SQL Server Management Studio (SSMS).

No Microsoft SQL Server Management Studio, foi criado o banco de dados 'JJBBIKE'.

Obs.: No desenvolvimento do código SQL, será utilizado o padrão de nomenclatura Snake Case.

1. Foi criado o banco de dados *Relational* (Relacional), que serve como origem para os dados;
2. As tabelas operacionais *sellers*, *products*, *customers*, *sales* e *sale_items* foram criadas no banco de dados *Relational*.
 - Assim como no ambiente analítico, as chaves primárias das tabelas relacionais foram configuradas com a propriedade *IDENTITY*, garantindo auto-incremento nativo pelo SGBD;
 - Todas as colunas, exceto as Primary Keys e *validity_end_date*, foram definidas com a restrição *NOT NULL*, garantindo a integridade dos dados no nível do banco de dados;
 - As Foreign Keys (FKs) foram implementadas para garantir a consistência dos dados entre as tabelas:
 - As tabelas *sales* e *sale_items* possuem relacionamentos rígidos com as tabelas de cadastro (*customers*, *sellers* e *products*), impedindo a criação de registros órfãos no sistema;
 - A restrição padrão ao definir uma FK sem especificar uma ação de exclusão ou atualização é o *NO ACTION*, que, caso se tente deletar um registro na tabela pai que possua dependentes na tabela filha, bloqueia a operação e gera um erro, impedindo a criação de dados órfãos.
3. Foram feitos inserts nas tabelas *sellers*, *products*, *customers*, *sales* e *sale_items*.
 - **Obs.:** O comando *SET DATEFORMAT dmy* instrui o banco de dados que o formato de data enviado é Dia/Mês/Ano, portanto, por exemplo, para 25/04/2026, o SGBD compreende corretamente que 25 é o dia e 04 é o mês de abril, executando o *INSERT* com sucesso.

Arquivos utilizados (1. Environments/1. OLTP/):

- **Pontos 1 e 2:** 1. *Scripts_Create_Table_Relational.sql*;
- **Ponto 3:** 2. *Insert_Into_Customers.sql*, 3. *Insert_Into_Products.sql*, 4. *Insert_Into_Sellers.sql*, 5. *Insert_Into_Sales.sql* e 6. *Insert_Into_SaleItems.sql*.

4. Ambiente Analítico (OLAP) - Microsoft SQL Server Management Studio (SSMS).

No Microsoft SQL Server Management Studio, foi utilizado o banco de dados 'JJBIKE'.

Obs.: No desenvolvimento do código SQL, será utilizado o padrão de nomenclatura *snake_case*.

4.1. Banco de Dados Analítico (OLAP)

1. Foi criado o banco de dados *Dimensional* (Dimensional), que representa o ambiente analítico (Data Warehouse (DW)) estruturado em um modelo Star Schema;
2. As dimensões *dim_seller*, *dim_customer*, *dim_product*, *dim_time* e a tabela fato *fact_sales* foram criadas no banco de dados *Dimensional*.
 - A propriedade *IDENTITY* no SQL Server é utilizada para gerar automaticamente as SKs como chaves primárias internas do DW em uma sequência crescente e incremental;
 - Todas as colunas, exceto as Primary Keys e *validity_end_date*, foram definidas com a restrição *NOT NULL*, garantindo a integridade dos dados na camada analítica;

- Foi aplicado o conceito de Slowly Changing Dimensions (SCD) Tipo 2 nas dimensões de vendedores, clientes e produtos por meio das colunas *validity_start_date* e *validity_end_date*, permitindo o versionamento completo e a preservação do histórico dos dados ao longo do tempo.

3. Foi realizada a carga da dimensão tempo. Esse processo foi realizado através de três estruturas principais:

- O bloco *DECLARE* define onde a contagem começa, onde termina e o passo de incremento:
 - **@start_date** (data inicial) e **@end_date** (data final): O script mapeia o período de 1º de janeiro de 1970 a 31 de dezembro de 2030. (O formato YYYYMMDD é utilizado por segurança, pois o SQL Server o lê corretamente independentemente do idioma do servidor);
 - **@step = 1**: Indica que o avanço será feito 1 dia por vez.
- O bloco *WITH date_sequence* (sequência de datas) *AS* utiliza um recurso chamado CTE Recursiva (Common Table Expression) para criar o seguinte laço:
 - **WITH date_sequence (CTE Recursiva)**: Funciona como um gerador automático de calendário, alimentando o código com uma tabela virtual contendo todas as datas possíveis entre as datas de início e fim definidas;
 - **SELECT**: Fatia cada uma dessas datas geradas para extrair e mapear informações úteis para a análise, como dia, mês, ano, dia da semana e trimestre;
 - **OPTION (MAXRECURSION 0)**: Garante que o banco de dados ignore o limite padrão de recursão, permitindo que o laço interno processe e insira todos os milhares de datas de uma só vez em perfeita ordem cronológica.
- O bloco *ORDER BY 1* garante que a tabela de dimensão tempo seja inserida em ordem cronológica perfeita, do dia mais antigo para o mais novo.

4. As seguintes cargas de dados do banco de dados transacional (*Relational*) para o banco de dados analítico (*Dimensional*) foram realizadas na ordem descrita:

- **Carga Inicial de clientes, produtos e vendedores:**
 - **Auxiliares de data**: O código começa declarando duas variáveis de tempo globais (*@dt_end* (data fim) e *@dt_start* (data início)) com uma diferença de 3 milissegundos entre elas. Elas garantem que todas as linhas processadas no mesmo lote compartilhem exatamente os mesmos timestamps, evitando inconsistências no DW;
 - **Lógica do MERGE (SCD Tipo 2)**: Em vez de usar CTEs separadas para atualizar e inserir, o processo utiliza a instrução nativa *MERGE*, comparando a tabela de origem (*USING*) com a dimensão de destino (*AS T*). O join é feito combinando o ID da origem com o filtro *T.validity_end_date IS NULL*, garantindo que a comparação ocorra apenas contra o histórico atualmente ativo;
 - **Tratamento de alterações e novos registros**: Se o ID coincide mas algum atributo (como *seller_name* (nome do vendedor), *customer_state* (estado do cliente), *customer_gender* (gênero do cliente) ou *customer_status* (status do cliente)) mudou, o *MERGE* executa um *THEN UPDATE* definindo a data de fim como *@dt_end* (fechando o histórico antigo). Se o ID não é encontrado no destino (*WHEN NOT MATCHED BY TARGET*), o *MERGE* executa automaticamente um *THEN INSERT* criando o registro com data de início igual a *@dt_start* e data de fim como *NULL*;

- **OUTPUT (geração do novo histórico):** Para completar a lógica do SCD Tipo 2 (que requer duas ações para registros alterados), a cláusula *OUTPUT* captura o *\$ACTION* de tudo que aconteceu no bloco interno. Um *INSERT INTO ... SELECT* externo envolve o *MERGE* e, usando o filtro *WHERE O.action = 'UPDATE'*, captura apenas as linhas que foram fechadas e insere uma nova versão ativa com dados atualizados e data de início em *@dt_start*.
- **Carga da fato vendas (janeiro, fevereiro, março e demais meses):**
 - **Lógica de inserção:** O código utiliza um *INSERT INTO ... SELECT* que consolida os dados transacionais diretamente na tabela fato;
 - **Mapeamento de chaves (SKs):** Realiza *INNER JOIN* entre as tabelas relacionais *sales* e *sale_items* e as dimensões do DW para capturar as SKs e atributos de métricas. A regra crucial aplicada em cada join dimensional é o filtro *validity_end_date IS NULL*. Isso garante que a venda seja associada à versão de SK do cliente, produto ou vendedor que estava ativa e válida no exato momento em que a carga foi executada;
 - **Filtro temporal:** Utiliza a função *MONTH(v.sale_date)* na cláusula *WHERE* para segmentar e isolar a carga da fato mês a mês (Janeiro = 1, Fevereiro = 2, Março = 3 e demais meses com > 3).
- **Atualização de Status (Simulação de SCD Tipo 2):**
 - **Alteração relacional:** Um *UPDATE* simples é feito diretamente na tabela de origem (*Relational..customers* (Relacional..customers)) simulando uma mudança real nos dados;
 - **Processamento de histórico:** Ao rodar o mesmo bloco de carga de clientes novamente, as variáveis de data são redeclaradas devido ao escopo de lote (*GO*). O *MERGE* detecta que *customer_status* da origem difere do *customer_status* armazenado em *dim_customer*. Isso aciona a condição *MATCHED*, fechando o registro antigo com a data de fim e passando o registro alterado para o *OUTPUT*, que cria a nova linha ativa preservando o histórico perfeito do que o cliente era antes de se tornar 'Gold' ou 'Platinum'.
- **Verificação de Resultados:**
 - **Consultas de Auditoria:** O roteiro finaliza com consultas *SELECT* combinando *fact_sales* com *dim_customer* e *dim_time*. Isso prova fisicamente o comportamento do SCD Tipo 2: as vendas de janeiro continuam associadas às SKs antigas (snapshot daquele momento), enquanto as de fevereiro em diante já apontam para a nova SK com o status atualizado.

Arquivos utilizados (1. Environments/2. OLAP):

- **Pontos 1 e 2:** 1. *Scripts_Create_Table_Dimensional.sql*;
- **Ponto 3:** 2. *Insert_Into_dim_time.sql*;
- **Ponto 4:** 3. *Script.sql*.

4.2. KPI.

O Microsoft SQL Server Management Studio (SSMS) não é um servidor de dados dimensional, portanto foi criada uma tabela desnormalizada (uma única tabela com todos os dados) para medir a receita total da empresa fictícia JJBike. Esse processo foi realizado através de quatro estruturas principais:

- O bloco *SELECT* seleciona apenas as colunas de interesse para a análise macro, extraindo o atributo temporal (mês), calculando a métrica agregada de receita total (*SUM*) da tabela fato e gerando dinamicamente a coluna de metas por meio de uma estrutura condicional (*CASE*);
- A função de conversão *CAST(... AS NUMERIC(18,2))*, que envolve todo o bloco *CASE*, substitui o operador *::numeric* do PostgreSQL. É utilizada para forçar e padronizar o tipo de dado da coluna de metas como um valor decimal exato com duas casas decimais, garantindo compatibilidade matemática perfeita com a métrica de receita ao calcular os alvos de atingimento;
- O bloco *INTO* define a criação física da nova tabela no banco de dados, especificando *Dimensional..kpi* (*Dimensional..kpi*) como destino final que armazenará os dados consolidados de resultados e metas por mês;
- O bloco *FROM* conecta a tabela central *Dimensional..fact_sales* (*Dimensional..fact_sales*) à dimensão *Dimensional..dim_time* (*Dimensional..dim_time*) por meio de um *INNER JOIN* utilizando as chaves substitutas (SKs), agrupando (*GROUP BY*) e ordenando (*ORDER BY*) os resultados finais por mês.

Arquivos utilizados (1. Environments/2. OLAP):

- 4. *KPI.sql*.

5. Relatórios - Power BI

1. As tabelas do banco de dados *Dimensional*, criadas no Microsoft SQL Server Management Studio, foram conectadas ao Power BI.

2. Foram feitas as seguintes alterações nas tabelas:

- Em *fact_sales*, foi criada uma coluna chamada *discount_percent* (percentual de desconto) (com duas casas decimais), com a seguinte fórmula: *discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2)*;
- Em *dim_customer*, foi criada uma coluna chamada *full_location* (localização completa), com a seguinte fórmula:
 - *location_map = SWITCH('dimensional dim_customer'[customer_state], "AC", "Acre", "AL", "Alagoas", "AM", "Amazonas", "AP", "Amapá", "BA", "Bahia", "CE", "Ceará", "DF", "Distrito Federal", "ES", "Espírito Santo", "GO", "Goiás", "MA", "Maranhão", "MG", "Minas Gerais", "MS", "Mato Grosso do Sul", "MT", "Mato Grosso", "PA", "Pará", "PB", "Paraíba", "PE", "Pernambuco", "PI", "Piauí", "PR", "Paraná", "RJ", "Rio de Janeiro", "RN", "Rio Grande do Norte", "RO", "Rondônia", "RR", "Roraima", "RS", "Rio Grande do Sul", "SC", "Santa Catarina", "SE", "Sergipe", "SP", "São Paulo", "TO", "Tocantins", "Unknown") & ", Brazil".*
- Em *dim_kpi*, foram criadas três métricas para que, no relatório de KPI, o card tenha comportamento dinâmico de acordo com o que estiver selecionado nos botões do button slicer:
 - *measure_target =*

```

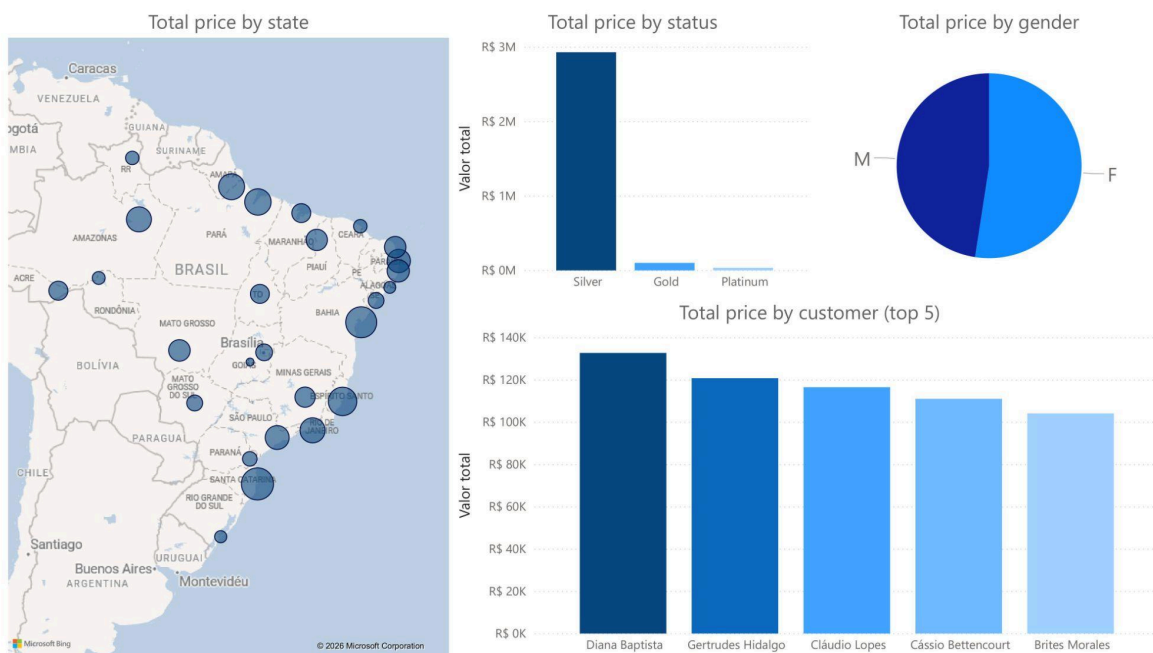
IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
    dim_time'[time_year]),
    SUM('dimensional kpi'[target]),
    0
).
○ measure_total_revenue =
IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
    dim_time'[time_year]),
    SUM('dimensional kpi'[total_revenue]),
    0
).

```

3. Foram criados os seguintes relatórios:

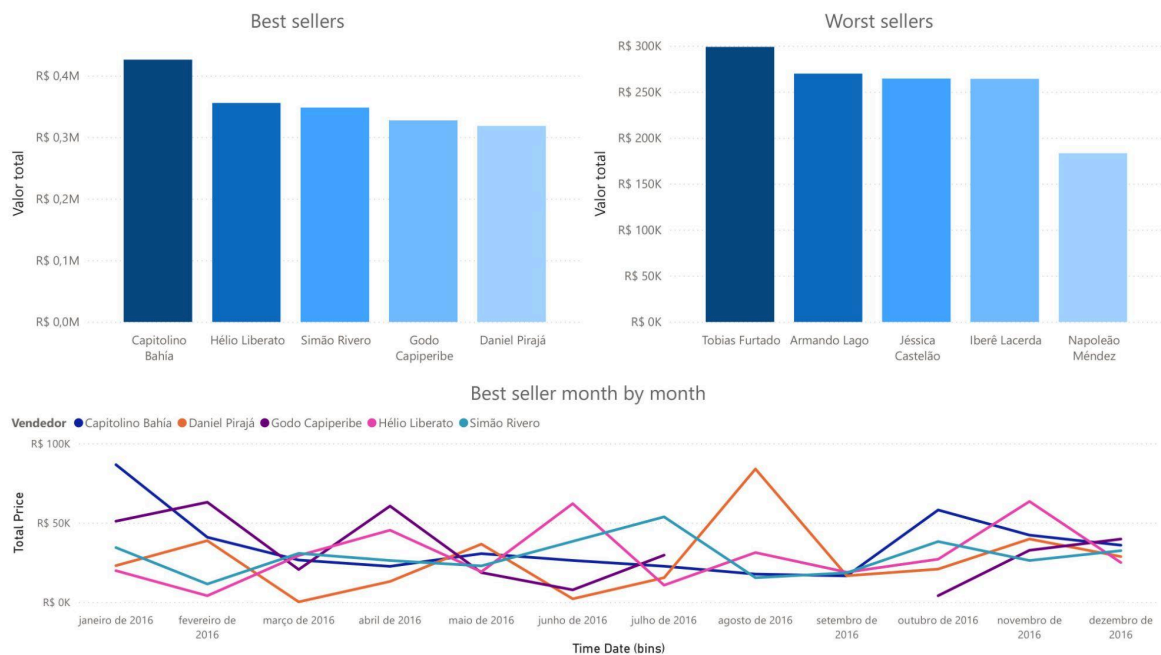
(Clientes)

O relatório de clientes apresenta: somatório de valor total por estado, somatório de valor total por status, somatório de valor total por gênero e somatório de valor total por cliente (top 5).



(Vendedores)

O relatório de vendedores apresenta: somatório de valor total por vendedor (top 5 melhores vendedores, top 5 piores vendedores e top 5 melhores vendedores comparados mês a mês).



(Produtos)

O relatório de produtos apresenta: somatório de valor total por produto (top 5 de produtos mais vendidos e top 5 de produtos menos vendidos) e somatório de desconto por produto (top 5 de produtos com mais desconto acumulado e top 5 de produtos com menos desconto acumulado).



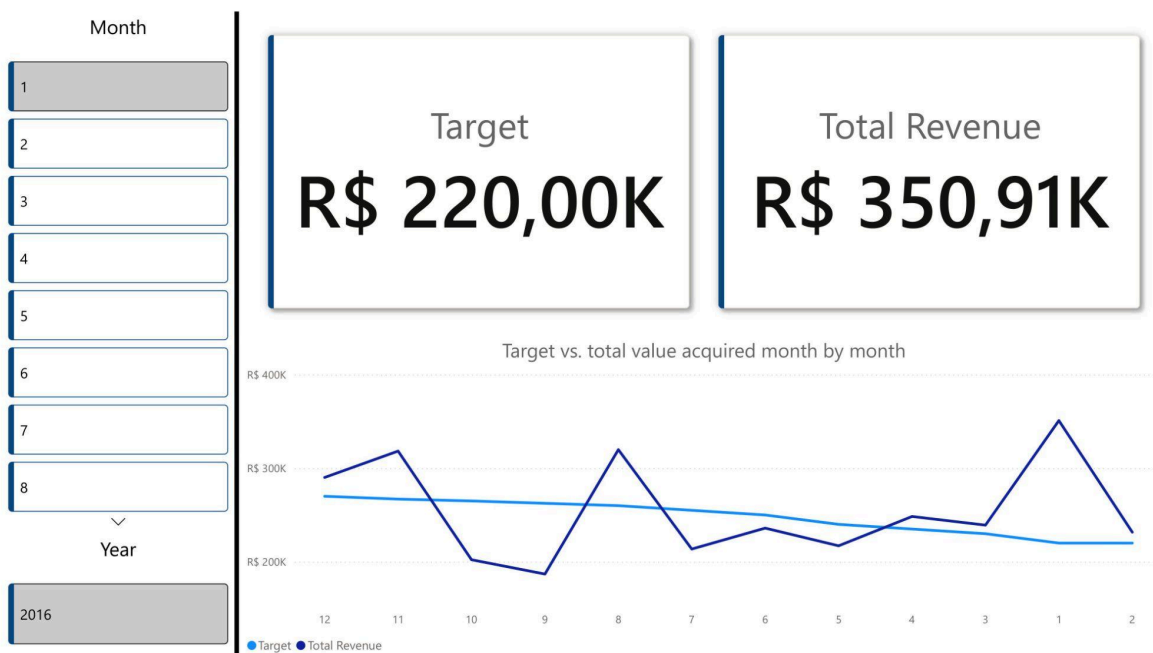
(Vendas)

O relatório de vendas apresenta: somatório de valor total de produtos vendidos (comparados mês a mês), somatório de quantidade de produtos vendidos (comparados mês a mês), somatório de desconto de produtos vendidos (comparados mês a mês) e cards de produtos, vendedores e clientes para filtragem.



(KPI)

O relatório de KPI apresenta: meta, valor total adquirido, somatório de meta e valor total adquirido (comparados mês a mês) e cards de mês e ano para filtragem.



Arquivos gerados disponíveis em '2. Artifacts/1. Reports'.

6. Cubo - Microsoft Visual Studio.

1. As tabelas do banco de dados *Dimensional* foram estruturadas e hospedadas em uma instância completa do SQL Server (Developer Edition), garantindo suporte nativo aos recursos de Business Intelligence (BI);

2. Foi criada uma Data Source View (DSV), um ambiente visual onde as tabelas fato e dimensões foram mapeadas (excluindo a tabela *kpi* (kpi)), gerando um modelo lógico no formato Star Schema;

3. Foram criadas as seguintes dimensões:

- **dim_customer**: *customer_key* (chave cliente), *customer_name* (nome do cliente), *customer_gender*, *customer_state* e *customer_status*;
- **dim_product**: *product_key* (chave produto) e *product_name* (nome do produto);
- **dim_time**: *time_key* (chave tempo), *time_year*, *time_quarter*, *time_month* e *time_day*;
- **dim_seller**: *seller_key* (chave vendedor) e *seller_name*.

4. Foram criadas as seguintes hierarquias a partir das dimensões:

- **Geographic Hierarchy** (Hierarquia Geográfica): *customer_state* → *customer_name* (*dim_customer*);
- **Time Hierarchy** (Hierarquia Temporal): *time_year* → *time_quarter* → *time_month* → *time_day* (*dim_time*);
- Observações:
 - *customer_gender* e *customer_status* (de *dim_customer*) devem ficar livres no painel lateral como Atributos de Dimensão Soltos, pois inseridos na *Geographic Hierarchy* implicaria que são parte de uma subdivisão geográfica;
 - *dim_seller* e *dim_product* possuem apenas um atributo além de suas chaves, portanto não geraram hierarquias e seus atributos tornaram-se Atributos de Dimensão Soltos;
 - Para evitar erros de chave duplicada no mecanismo do Analysis Services durante o processamento de *dim_time*, as propriedades KeyColumns foram customizadas como chaves compostas, vinculando *time_quarter* com *time_year*, *time_month* com *time_year*, e *time_day* com *time_month* e *time_year*, garantindo a unicidade e integridade da árvore de relacionamento de atributos.

5. O cubo foi estruturado a partir da tabela fato *fact_sales*. Durante o processo, as chaves de relacionamento e campos analiticamente incorretos (como *unit_price* (preço unitário), cuja agregação por soma não faz sentido de negócio) foram ignorados. Apenas as métricas reais de desempenho foram mantidas (*quantity* (quantidade), *total_price*, *discount* (desconto) e a contagem automática de registros);

6. Foi realizado um deploy bem-sucedido do projeto para a instância local do servidor Analysis Services, compilando as estruturas e carregando os dados em memória;

7. Representação visual do cubo criado:

Customer State	Customer Name	Time Year	Time Quarter	Time Month	Time Day	Quantity	Discount	Total Price
AC	Alarico Quinterno	2016	4	12	10	1	60,45	155
AC	Antônio Sobral	2016	3	9	17	1	1593,36	8852
AC	Antônio Sobral	2016	4	10	16	2	1741,25	7935,6
AC	Basilio Soares	2016	1	3	29	1	4,65	155
AC	Basilio Soares	2016	2	4	17	9	879,08	22167,65
AC	Basilio Soares	2016	2	4	18	3	125,1	3289
AC	Basilio Soares	2016	2	5	14	1	64,07	2135,52
AC	Basilio Soares	2016	2	6	6	1	650,93	6509,3
AC	Basilio Soares	2016	2	6	24	1	16,92	188
AC	Basilio Soares	2016	3	7	5	2	775,25	7793
AC	Carla Briones	2016	4	12	5	1	587,75	2260,58
AC	Ester Castanho	2016	3	9	26	5	2327,33	15416,98
AC	Evaristo Bahia	2016	4	10	10	3	692,17	4836,7
AC	Irene Villanueva	2016	4	11	12	3	1654,34	7183,75
AL	Cid Pardo	2016	3	8	21	3	1769,02	13295
AL	Cid Pardo	2016	4	10	18	3	2430,31	13522,61
AM	Deise Farias	2016	3	8	22	3	1231,04	8976,8
AM	Deise Farias	2016	3	9	7	1	1472,16	9201
AM	Diana Baptista	2016	1	2	12	4	79,67	11112,58
AM	Diana Baptista	2016	1	2	12	1	1,60	160,2

Arquivos gerados disponíveis em '2. Artifacts/2. Cube/'.